Module 3: AI for Mechatronics

Convolutional Neural Networks: Concepts, Engineering Applications and Practical Transfer Learning

Dr. Rajan Prasad

SMEC
VIT University

Fall 2026

Lecture Roadmap

- Understand why CNNs are preferred for image-like data.
- Learn the basic building blocks of CNNs.
- Connect CNNs to mechatronics applications.
- Use pretrained CNNs such as VGG, ResNet, MobileNet and YOLO.
- Build a complete transfer-learning workflow from data to deployment.

Why CNNs?

Can you distinguish between these visually similar objects?

Similar colours, textures, edges, and shapes can make image classification challenging.

Reference: https://medium.com/eliza-effect/ai-ml-and-deep-learning-whats-the-difference-ba6bb2223589

Why CNNs?

Can you distinguish between these visually similar objects?

Similar colours, textures, edges, and shapes can make image classification challenging.

Reference: https://medium.com/eliza-effect/ai-ml-and-deep-learning-whats-the-difference-ba6bb2223589

Why CNNs?

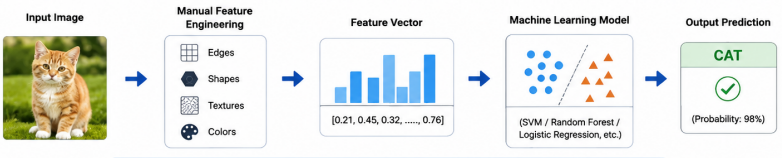
Can you distinguish between these visually similar objects?

Similar colours, textures, edges, and shapes can make image classification challenging.

Reference: https://medium.com/eliza-effect/ai-ml-and-deep-learning-whats-the-difference-ba6bb2223589

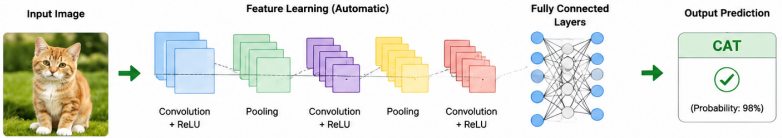
WHY DO WE NEED CNNs?

1. TRADITIONAL MACHINE LEARNING APPROACH



Key Idea: Humans extract features → Model learns from features → Predicts the class

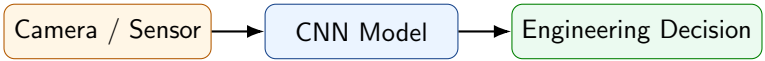
2. DEEP LEARNING APPROACH (CNN)



Key Idea: CNNs automatically learn hierarchical features (edges → textures → parts → objects) from raw images and make accurate predictions.

Takeaway: Traditional ML depends on manual feature engineering.
CNNs learn the right features automatically, making them more powerful and generalizable.

Why do mechatronics engineers need CNNs?



CNNs convert visual data into useful decisions for robots, machines and automation systems.

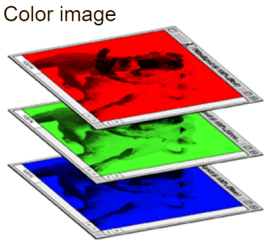
Image as a Matrix

A grayscale image is a matrix of pixel intensities.

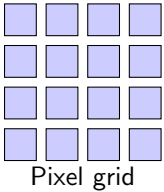
$$I = \begin{bmatrix} 12 & 20 & 35 & 40 \\ 18 & 25 & 44 & 50 \\ 21 & 30 & 60 & 70 \\ 28 & 38 & 65 & 80 \end{bmatrix}$$

$$Image \in \mathbb{R}^{H \times W \times 3}$$

- H: height
- W: width
- 3 channels: red, green, blue
- Each number represents brightness.
- Low value means dark pixel.
- High value means bright pixel.



Color image

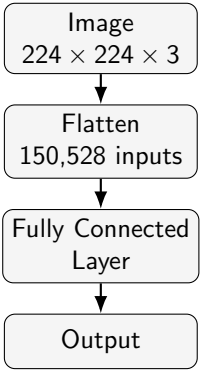


Why Do We Need CNN?

Problem: A normal fully connected neural network does not understand image structure.

MLP view of image

- Image is flattened into one long vector.
- Neighboring pixel information is not explicitly used.
- Every input connects to every neuron.



For images, the arrangement of pixels is important. CNNs preserve and exploit this arrangement.

Limitation of MLP for Images: Parameter Count

For a color image of size $224 \times 224 \times 3$,

$$224 \times 224 \times 3 = 150{,}528$$

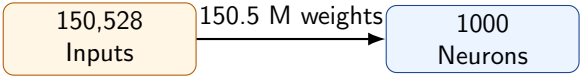
input values are required. If the first hidden layer has only 1000 neurons,

$$150{,}528 \times 1000 = 150{,}528{,}000$$

parameters are needed only in the first layer.

Main problems

- Very large memory requirement
- Slow training
- High overfitting risk
- Poor use of spatial structure



Why CNN Works Better for Images

CNNs use three important ideas.

Local spatial relation

A filter looks at a small neighborhood of pixels instead of the full image at once.

Weight sharing

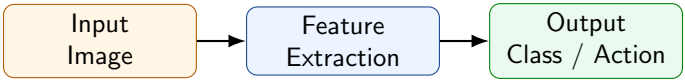
The same filter is reused across the image, reducing parameters.

Feature hierarchy

Early layers learn edges. Later layers learn parts and objects.

What Is a CNN?

Definition: A Convolutional Neural Network (CNN) is a deep learning model designed to process image-like data by learning spatial features using convolution filters.



- The input may be an image, thermal map, depth image or spectrogram.
- The CNN learns useful filters automatically during training.
- The output may be a class label, defect status, object position or robot command.

CNN: All WH Questions

Question	Answer
What is CNN?	A neural network for image-like and spatial data.
Why use CNN?	It captures local patterns with fewer parameters than an MLP.
When to use CNN?	When nearby values contain useful information (strongly related).
Where is CNN used?	Robot vision, inspection, driving, medical imaging and automation.
How does CNN work?	Convolution, activation, pooling and classification layers.
Which model?	VGG, ResNet, MobileNet, EfficientNet, DenseNet or YOLO depending on task and hardware.

When Should We Use CNN?

CNNs are useful when the input has a meaningful spatial layout.

Good use cases

- Camera images
- Thermal images
- Depth images
- Spectrograms from vibration or audio
- Video frames

Usually not the first choice

- Excel-like tabular data
- Independent sensor values
- Text sequences
- Pure time-series forecasting

Rule of thumb: If local neighboring patterns matter, think CNN.

CNN versus MLP, RNN and Transformer

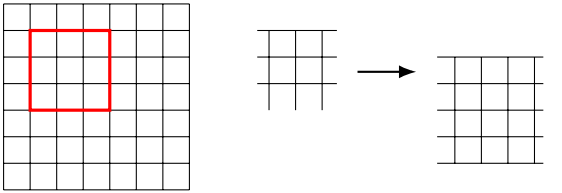
Model	Best for	Example in mechatronics
MLP	Tabular data	Predict motor torque from sensor readings.
CNN	Image-like data	Detect surface defects from camera images.
RNN/LSTM/GRU	Time sequences	Predict vibration trend or motor current sequence.
Transformer	Large sequence or vision tasks	Advanced vision-language or long time-series models.

CNN is the natural starting point for most visual perception tasks in mechatronics.

How does a CNN actually work?

How Convolution Works

A convolution filter slides over the image and produces a feature map.



$$\text{Feature Map} = \text{Image} * \text{Kernel}$$

- The kernel contains learnable weights.
- During training, CNN learns filters useful for the task.

Numerical Convolution Example

Consider a 3×3 image patch and a 3×3 kernel.

$$X = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 3 \\ 2 & 1 & 0 \end{bmatrix} \qquad K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} \qquad Y = \sum X_{ij} K_{ij}$$

$$Y = (1)(1) + (2)(0) + (1)(-1) + (0)(1) + (1)(0) + (3)(-1) + (2)(1) + (1)(0) + (0)(-1)$$

$$Y = 1 + 0 - 1 + 0 + 0 - 3 + 2 + 0 + 0 = -1$$

Repeating this calculation over the image creates a feature map.

Different filters respond to different visual patterns.

Vertical edge

$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

Horizontal edge

$$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Sharpen

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

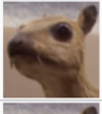
- In a trained CNN, these filters are learned automatically.
- Early layers often learn edges and simple textures.
- Deeper layers learn meaningful object parts.

Convolution Operation

Example

Input image

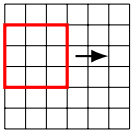
Operation	Filter	Convolved Image
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	

Operation	Filter	Convolved Image
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur (approximation)	$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

Stride and Padding

Stride

- Step size of the moving filter.
- Larger stride reduces output size.
- Useful for downsampling.



Stride = 2

Padding

- Adds border pixels around the image.
- Helps preserve spatial size.
- Allows edge information to be processed.

$$O = \frac{N - F + 2P}{S} + 1$$

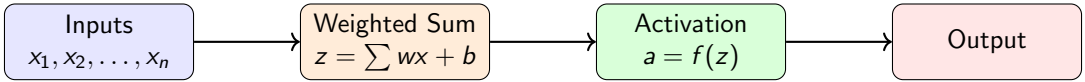
where N is input size, F filter size, P padding, S stride, and O is the output size dimension.

What Are Activation Functions?

An activation function is a mathematical function applied to the output of a neuron. It decides how strongly the neuron should be activated.

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

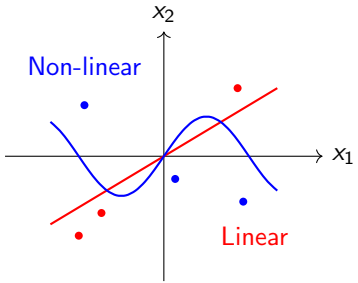
$$a = f(z)$$



- Without activation, a neuron only gives a linear output.
- Activation functions introduce non-linearity into neural networks.
- Common examples are Sigmoid, Tanh, ReLU, Leaky ReLU and Softmax.

Why Do We Need Activation Functions?

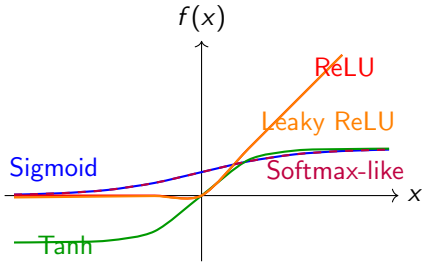
- Neural networks must learn complex real-life patterns.
- A purely linear network cannot model curved or complicated boundaries.
- Activation functions allow the network to learn non-linear relationships.
- They help decide whether information should pass strongly, weakly or not at all.
- Different tasks use different activation functions.



Linear layers only $\Rightarrow$ limited learning Activation functions $\Rightarrow$ complex learning

Common Activation Functions

Function	Expression	Typical Use
Sigmoid	$\frac{1}{1+e^{-x}}$	Binary output
Tanh	$\frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$	Older RNN hidden layers
ReLU	$\max(0, x)$	CNN/MLP hidden layers
Leaky ReLU	$\max(0.01x, x)$	Avoid dead ReLU
Softmax	$\frac{e^{z_i}}{\sum_j e^{z_j}}$	Multiclass output



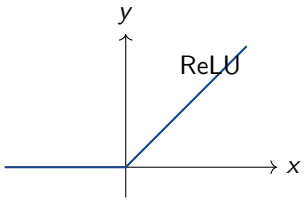
Activation Function: ReLU

After convolution, CNN applies a nonlinear activation function.

$$\mathrm{ReLU}(x) = \max(0, x)$$

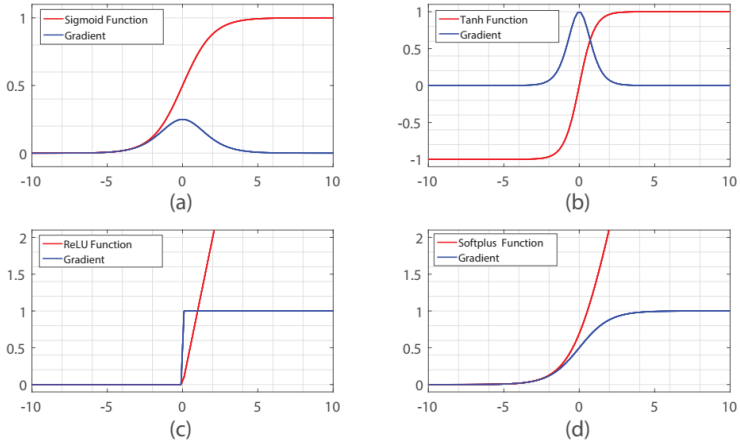
Why ReLU [Rectified Linear Unit]?

- Simple and fast
- Adds nonlinearity
- Helps train deep networks
- Reduces vanishing gradient compared with sigmoid in many deep networks



Why to prefer ReLU

Reduces Vanishing Gradient.



Ranking to Learn and Learning to Rank: On the Role of Ranking in Pattern Recognition Applications: https://arxiv.org/pdf/1706.05933

Pooling Layer

Pooling reduces spatial size and makes features more robust.

Max pooling

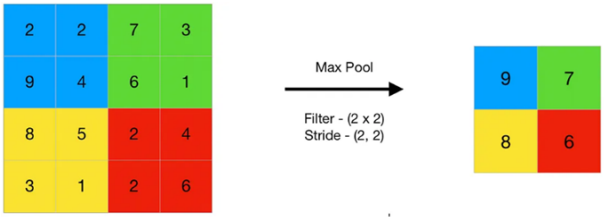
$$\begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix} \rightarrow 4$$

- Keeps strongest response.
- Common in CNNs.

Average pooling

$$\begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix} \rightarrow 2.5$$

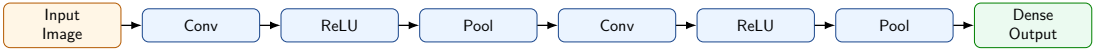
- Keeps average response.
- Smooths feature maps.

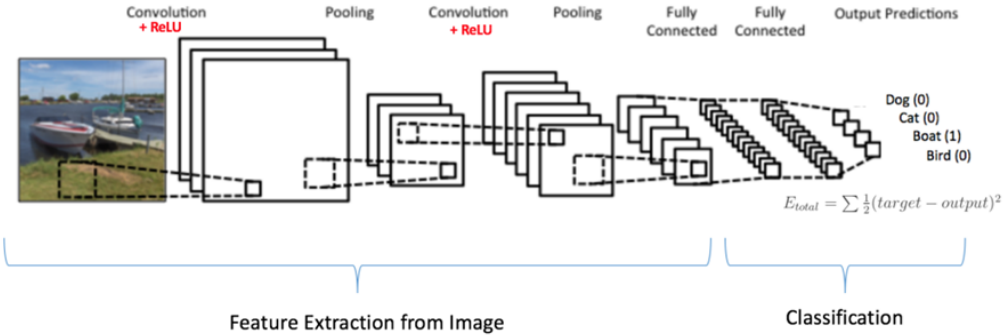


Reduces computation and gives some tolerance to small shifts in object position.

CNN Building Blocks

A basic CNN combines convolution, activation, pooling and dense layers.



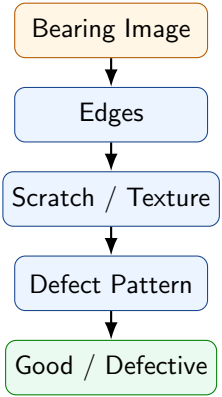


Convolution	**ReLU**	**Pooling**	**Dense**
Extract Features	Add Nonlinearity	Downsample	Prediction

How CNN Learns: A Bearing-Inspection Story

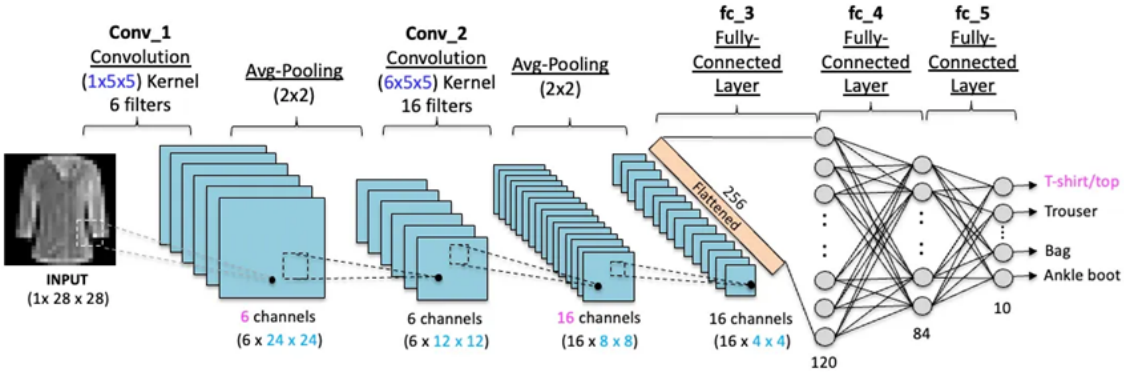
What does the CNN "see"?

1. At first, it receives only pixel values.
2. Early filters detect edges and intensity changes.
3. Middle layers combine edges into scratches and textures.
4. Deeper layers recognize meaningful defect patterns.
5. The final layer predicts **good** or **defective**.



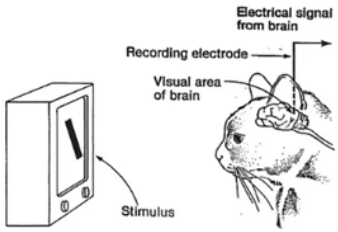
A CNN is not explicitly instructed to "look for a scratch." Instead, it automatically learns useful features from many labeled examples during training.

How CNN Learns: Example



Reference: https://medium.com/@lmpo/the-evolution-of-convolutional-neural-networks-from-lenet-to-convnext-0b1c37ccb52f/

Hubel and Wiesel's Experiments (1950s):From biological vision to hierarchical feature learning



- Simple cells: detecting basic features
- Complex cells: combining simple features to detect more complex patterns.

Pixels → Edges → Textures → Parts → Object / Defect

Visual Cortex: neurons respond to edges and orientations

↓

Early CNN Layers: detect edges, lines and intensity changes

↓

Middle Layers: combine edges into textures, corners and patterns

↓

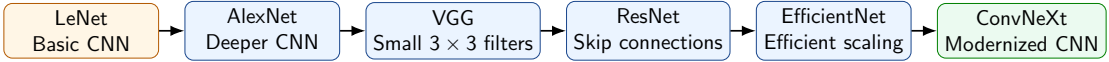
Deeper Layers: recognize object parts and defect structures

↓

Final Decision: object class or good/defective

Reference: https://medium.com/@lmpo/the-evolution-of-convolutional-neural-networks-from-lenet-to-convnext-0b1c37ccb52f

Evolution of CNN Architectures



What changed over time?

- Networks became deeper.
- Feature extraction became more powerful.
- Training became more stable.

Engineering objective

- Improve accuracy.
- Reduce computation and memory.
- Support practical deployment.

Key idea

Although architectures changed, the central principle remained the same: learn simple local features first and combine them into increasingly complex visual concepts.

Understanding Basic CNN Operations through an Example

What Will We Implement?

1. Represent a simple image as a numerical matrix.
2. Apply convolution using different kernels
3. Use an activation function
4. Apply pooling .
5. Flatten the resulting feature maps for classification.

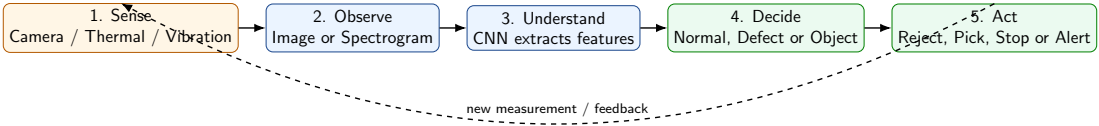
Learning Objective

Understand how convolution, activation, and pooling progressively transform an input image into useful features for classification.

Scan to Run the Colab Notebook

Open the notebook and execute each cell to observe how the image changes after every CNN operation.

How does a CNN become part of a real mechatronics system?



Automatic bearing inspection

A camera captures each bearing on a conveyor. The CNN detects scratches or surface defects. The controller then activates a pneumatic actuator to remove the defective bearing, while acceptable parts continue to the next station.

Sense → Understand → Decide → Act → Feedback

Benefits and Limitations of CNN

Benefits

- Automatic feature extraction
- High accuracy in vision tasks
- Fewer parameters than large MLPs
- Robust to small shifts
- Less manual feature engineering
- Works well with transfer learning

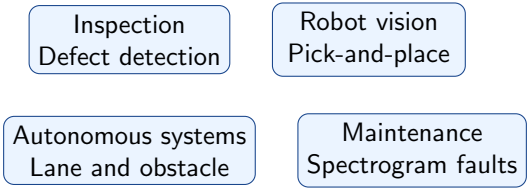
Limitations

- Needs labeled data
- Training can be computationally expensive
- May fail under unseen lighting or viewpoints
- Large models may be difficult for embedded hardware
- Interpretability can be limited

Try Yourself: PyTorch CNN Tutorial

`https://www.datacamp.com/tutorial/pytorch-cnn-tutorial`

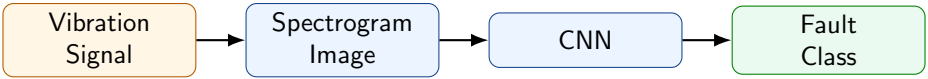
Where CNN Is Used in Mechatronics



- Industrial cameras can inspect bearings, gears, welds and PCBs.
- Robot cameras can identify objects and guide manipulation.
- Autonomous systems can detect lanes, signs and obstacles.
- Vibration signals can be converted to spectrograms and classified using CNNs.

Predictive Maintenance Using Spectrograms

A CNN can also work on image-like representations of signals.



- Raw vibration is a time-series signal.
- A spectrogram represents how frequency content changes with time.
- CNN learns patterns in the spectrogram to detect bearing or motor faults.

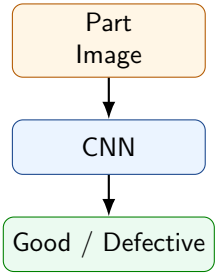
Industrial Defect-Detection Example

Problem

- Detect surface cracks, scratches or missing parts.
- Input: image from industrial camera.
- Output: good or defective.

CNN learns

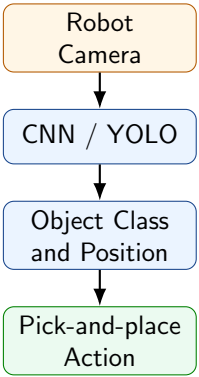
- Edges
- Texture changes
- Crack patterns
- Shape irregularities



Robot Pick-and-Place Example

Task

- Robot must identify a target object.
- Camera provides image or video frame.
- CNN detects object class or object position.
- Robot controller plans grasping action.



In robot vision, CNN output must be connected to a physical action.

How do engineers use CNNs in practice?

Why Not Train from Scratch?

Training a CNN from scratch is often unnecessary and inefficient.

Training from scratch needs

- Large labeled dataset
- Long training time
- Good GPU resources
- Careful hyperparameter tuning

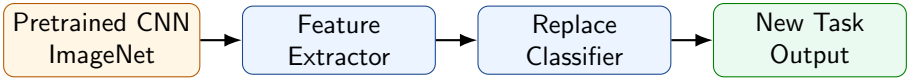
Engineering reality

- Dataset may have only hundreds of images.
- Time is limited.
- Hardware may be limited.
- Fast deployment is needed.

Most practical projects start with a pretrained model.

What Is Transfer Learning?

Transfer learning uses knowledge learned from a large dataset and adapts it to a new task.



- Keep early layers that already know generic visual features.
- Replace final classifier for your classes.
- Fine-tune some or all layers using your dataset.

Popular Pretrained Models

Model	Strength	Typical use
VGG16	Simple and easy to understand	Teaching, baseline classification.
ResNet	Strong accuracy and deep architecture	Industrial classification and transfer learning.
MobileNet	Lightweight and fast	Mobile robots and embedded systems.
EfficientNet	Good accuracy-size balance	High accuracy with fewer parameters.
DenseNet	Feature reuse	Classification with compact representations.
YOLO	Real-time detection	Object localization and detection in video.

How to Select a Pretrained Model

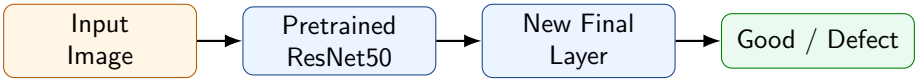
Choose the model based on task and deployment hardware.

Requirement	Suggested model
Highest accuracy	ResNet, EfficientNet.
Fast inference	MobileNet, EfficientNet-Lite.
Low memory	MobileNet, small EfficientNet variants.
Simple baseline	VGG16.
Object localization	YOLO, SSD or Faster R-CNN.
Embedded hardware	MobileNet or optimized YOLO variants.

There is no single best CNN. The best model depends on accuracy, speed, memory and hardware.

Using ResNet50 in Practice

ResNet uses residual connections that allow deeper networks to train effectively.



- Good general-purpose choice for engineering image classification.
- Often works well with small and medium datasets using transfer learning.
- Suitable for defect detection, object classification and quality inspection.

Using VGG16 in Practice

VGG16 is simple, deep and easy to explain.

Why use it?

- Clear structure
- Many 3×3 convolution filters
- Good teaching model
- Strong baseline for transfer learning

Limitations

- Large number of parameters
- Slower than MobileNet
- Not ideal for low-power embedded systems

Use VGG16 when simplicity and interpretability are more important than speed.

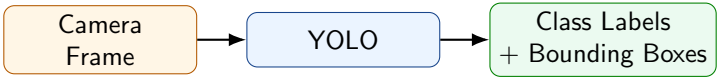
Using MobileNet for Embedded Mechatronics

MobileNet is designed for fast inference on limited hardware.

- Small model size
- Fast inference
- Lower power consumption
- Useful for mobile robots, drones and embedded inspection systems

Object Detection with YOLO

Classification tells **what** is in the image. Detection tells **what and where**.



Classification output

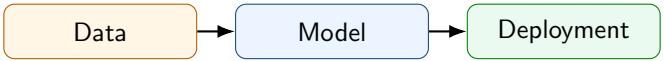
- Image contains bottle.

Detection output

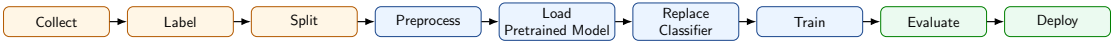
- Bottle at location (x, y, w, h).

YOLO is used for localization and detection, not only classification.

From dataset to deployed mechatronics system



Complete Transfer-Learning Workflow



Data preparation

- Collect application-specific images.
- Assign class labels or bounding boxes.
- Divide data into train, validation and test sets.

Model adaptation

- Resize and normalize the images.
- Load a pretrained CNN.
- Replace the original output layer.

Training and deployment

- Fine-tune the model.
- Evaluate on unseen test data.
- Export to the target hardware.

PyTorch Code Example: ResNet50 Transfer Learning

```python
import torch
import torch.nn as nn
from torchvision import models

# Load pretrained ResNet50
model = models.resnet50(weights="DEFAULT")

# Freeze feature extractor if dataset is small
for param in model.parameters():
param.requires_grad = False

# Replace final layer for two classes: good and defective
model.fc = nn.Linear(2048, 2)

# Train only the new classifier first
# Then optionally unfreeze deeper layers and fine-tune
```

Practical idea: use pretrained features and train only the part that is specific to your problem.

Evaluation Metrics

Accuracy alone may be misleading, especially when defects are rare.

Metric	Meaning
Accuracy	Fraction of total correct predictions.
Precision	Of predicted defects, how many are truly defects.
Recall	Of actual defects, how many were detected.
Confusion matrix	Shows correct and incorrect predictions for each class.

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}$$

Confusion Matrix for Defect Detection

	Predicted	
Actual	Good	Defective
Good	True Negative	False Positive
Defective	False Negative	True Positive

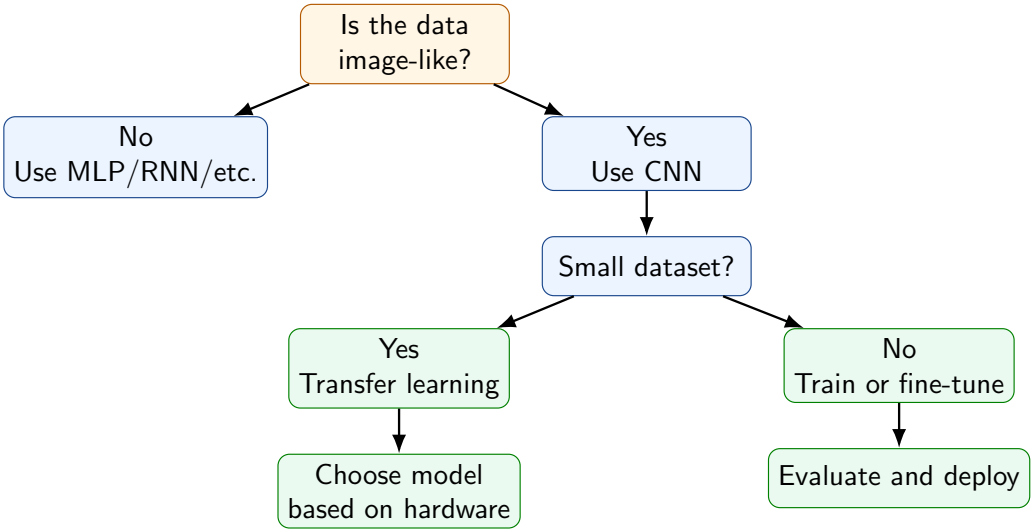
- **False positive:** good part rejected as defective.
- **False negative:** defective part accepted as good.
- In safety-critical manufacturing, false negatives may be more serious.

Deployment on Mechatronics Hardware

Hardware	Typical use
Laptop / GPU workstation	Training and testing large models.
NVIDIA Jetson	Edge AI for robots, drones and camera systems.
Raspberry Pi	Lightweight inference using small optimized models.
Industrial computer	Factory deployment with cameras and PLC integration.
Cloud server	Centralized training, monitoring and model updates.

A model that is accurate on a laptop may still be too slow for real-time embedded deployment.

CNN Decision Guide



Summary

- CNNs are designed for image-like data where local spatial patterns matter.
- Convolution, ReLU and pooling are the main building blocks.
- CNNs learn hierarchical features: edges, textures, parts and objects.
- In mechatronics, CNNs are used for inspection, robot vision, autonomous systems and predictive maintenance.
- In practice, engineers commonly use transfer learning with pretrained models.
- Model selection depends on accuracy, speed, memory and deployment hardware.

For most practical vision systems: start with a pretrained CNN, fine-tune it, evaluate carefully and deploy on suitable hardware.

Exercise

Exercise: Design a CNN-based inspection system for a conveyor belt with bearing.

1. What image data will you collect?
2. What labels are required?
3. Is the task classification, detection or segmentation?
4. Which pretrained model will you choose and why?
5. What metrics will you use?
6. Which hardware will you deploy on?

Discussion

Compare three choices: ResNet50, VGG16 and YOLO. Which is best for a real-time conveyor belt system?

Live Demo

Demo 1: Classification

- Dataset: good vs defective part images.
- Model: VGG16.
- Task: fine-tune final layer.
- Output: confusion matrix.

Demo 2: Detection [Not included]

- Dataset: webcam or sample object images.
- Model: YOLO.
- Task: detect and localize objects.
- Output: bounding boxes.

CNN making decisions on real images.

CNN Fundamentals Quiz

Instructions

- Scan the QR code to access the quiz.
- Sign in using your **VIT Google account**.
- The quiz contains **10 questions**.
- Only **one submission per account** is allowed.

CNN Fundamentals

Convolution • Kernel • Padding • Stride
Pooling • Feature Maps • CNN Architecture

QUIZ: Scan to begin

VGG-Based Transfer Learning for Bearing Fault Classification

What Will We Implement?

1. Load bearing-condition images.
2. Use a pretrained **VGG network** as the feature extractor.
3. Replace and train the final classification layers.
4. Evaluate the model using accuracy, predictions, and a confusion matrix.

Learning Objective

Understand how knowledge learned from a large image dataset can be transferred to classify bearing conditions using limited training data.

Scan to Run the Colab Notebook

Open the notebook, connect to the runtime, and execute each cell sequentially.

Thank You

Questions?